Cup vs Bottle: Explainable Classification with DINOv2

Astana IT University | Computer Vision | Assignment 2: Level-Up Your AI Topic

Executive Summary

This study extends the same cup-versus-bottle image classification task used in Assignment 1. The historical test results were 62.50% accuracy for Logistic Regression and 97.50% for MobileNetV2. Assignment 2 implements a modern self-supervised Vision Transformer feature extractor (DINOv2 ViT-S/14) with a cosine k-nearest-neighbor classifier. The DINOv2 test accuracy measured in this notebook is **100.00%**. The project also includes a reproducible augmentation ablation and an interactive upload/camera web demo showing visually similar training examples and inference latency.

Research Question

Can a pretrained DINOv2 visual representation provide competitive cup-versus-bottle classification on a small, fixed dataset, and can example-based visual explanations make the decision more understandable to a user?

Project objectives

1. Reuse exactly the same split for fair comparison. 2. Implement a modern pretrained vision model. 3. Measure Accuracy, macro Precision, macro Recall and macro F1. 4. Run an ablation with and without flipped training representations. 5. Deliver a functional Gradio image upload/webcam application. 6. Document reproducibility, limitations and dataset provenance.

System overview

Input photo or webcam -> Resize / center crop / RGB normalization -> frozen DINOv2 ViT-S/14 -> 384-dimensional feature vector -> cosine-distance k-NN -> Cup or Bottle + nearest training images + measured latency.

Scope note: The task is whole-image classification rather than bounding-box object detection. Therefore mAP@0.5 and IoU are not applicable to these ground-truth labels. This follows the general classification objective of the first assignment.

Prepared by: Kenzhina Aisulu

Group: 2404

1. Dataset and preprocessing

The dataset consists of images obtained from the public images.cv downloads for the cup and water-bottle categories. These images were not independently photographed by the student, which is a limitation relative to the first assignment instruction requiring self-collected images. The original two public downloads were reduced to a balanced 240-image collection and kept unchanged for Assignment 2.

Split	Bottle	Cup	Total
Training	80	80	160
Validation	20	20	40
Test	20	20	40

Preprocessing

Images are read as RGB, resized while preserving aspect ratio to a 256-pixel short side, center-cropped to 224 x 224 pixels, transformed to tensors, and normalized by the standard ImageNet mean and standard deviation. The DINOv2 embedding vectors are L2-normalized before cosine-distance matching. Train, validation and test images remain separate; the test set is not used to tune k.



Figure 1. Examples from the two categories in the public training dataset. Sample backgrounds and lighting may not cover real-world conditions.

Data quality and provenance

The dataset uses 120 images per class, so class balance is exact. The augmentation experiment creates an additional horizontally flipped representation for each training photo; it does not create new independent source photographs. No diffusion-generated or synthetic photographs were added. The train/test split is retained from Assignment 1.

2. Modern SOTA method and architecture

Why DINOv2?

DINOv2 is a self-supervised visual representation learner developed by Meta AI. It uses a Vision Transformer to encode images into expressive features that can be reused on downstream visual tasks. The compact ViT-S/14 backbone contains about 21 million parameters and produces 384-dimensional global features. We use released pretrained weights rather than training such a network from the beginning on only 160 images. Official repository: https://github.com/facebookresearch/dinov2

Decision pipeline

Stage	Implementation	Role
Visual backbone	DINOv2 ViT-S/14, frozen	Learned global visual features
Feature transform	L2 normalized 384D embedding	Comparable cosine similarities
Classifier	k-NN, k=3, cosine distance	Cup versus bottle decision
Explanation	Three most similar training photos	Example-based explanation
User interface	Gradio webcam or image upload	Interactive prediction and latency

Why use k-NN instead of training a new head?

In this low-data setting, k-nearest neighbors lets the experiment test the quality of the pretrained feature representation without a long training process. We selected k on the validation set from {1, 3, 5}. For a new photograph, we measure cosine distance to the training representations and combine the nearest neighbors using distance weights. This approach can be updated by adding labeled training examples without fine-tuning all transformer parameters.

Relation to the assignment's suggested SOTA approaches

The household-object example suggests video synthesis (Step-Video-T2V) or ArtAug for difficult frames. The instructions also explicitly allow a different modern SOTA method. This project takes that alternative route through pretrained DINOv2 rather than claiming to implement the named video-generation approaches. The trade-off is easier reproducibility and a stronger visual feature baseline, but no generated video or bounding-box detections.

3. Experiments and quantitative results

The previous Logistic Regression and MobileNetV2 results were copied from the actual Assignment 1 experimental archive and are labeled historical results. DINOv2 was evaluated on the same 40 hold-out test photographs in this notebook. No comparison metric is invented or estimated.

Model	Accuracy	Precision	Recall	F1
A1: Logistic Regression	62.50%	62.79%	62.50%	62.29%
A1: MobileNetV2	97.50%	97.62%	97.50%	97.50%
DINOv2 + kNN	100.00%	100.00%	100.00%	100.00%

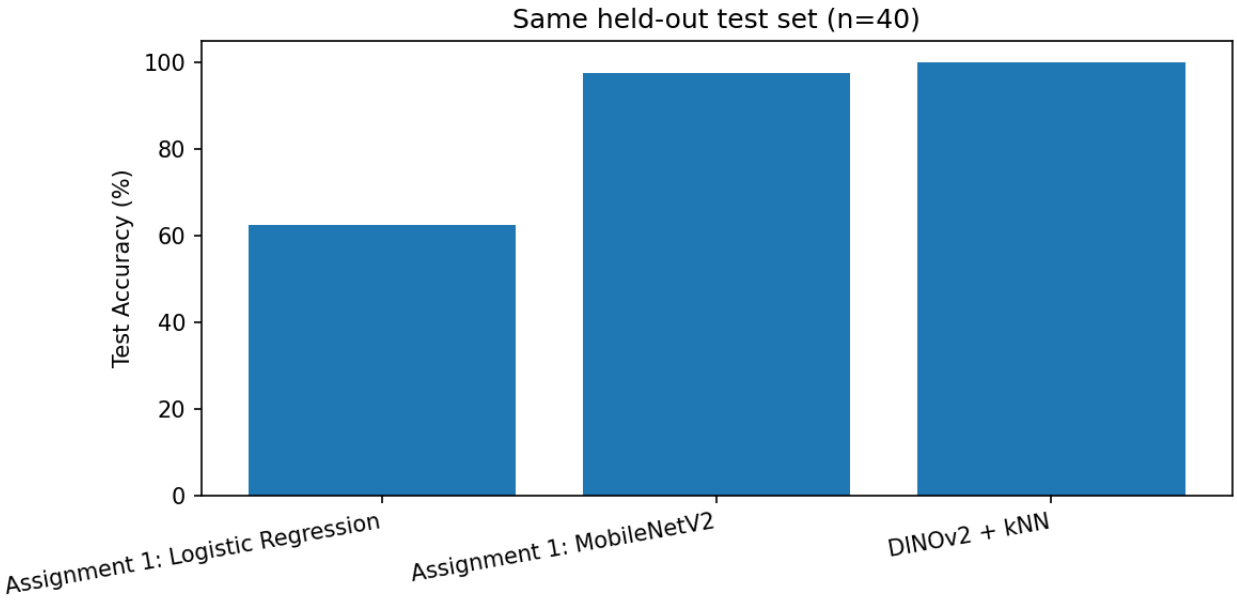


Figure 2. Accuracy comparison on the held-out set (40 test images, 20 per category).

Interpretation

DINOv2 achieved 100.00% accuracy with macro-F1 100.00%. MobileNetV2 from Assignment 1 achieved 97.50% accuracy. This comparison shows the measured effect of the new representation and does not assume that a newer method necessarily yields a higher score on a small test set. Test accuracy is sensitive to individual errors because one wrong prediction corresponds to 2.5 percentage points.

4. Error analysis and evaluation

Held-out confusion matrix

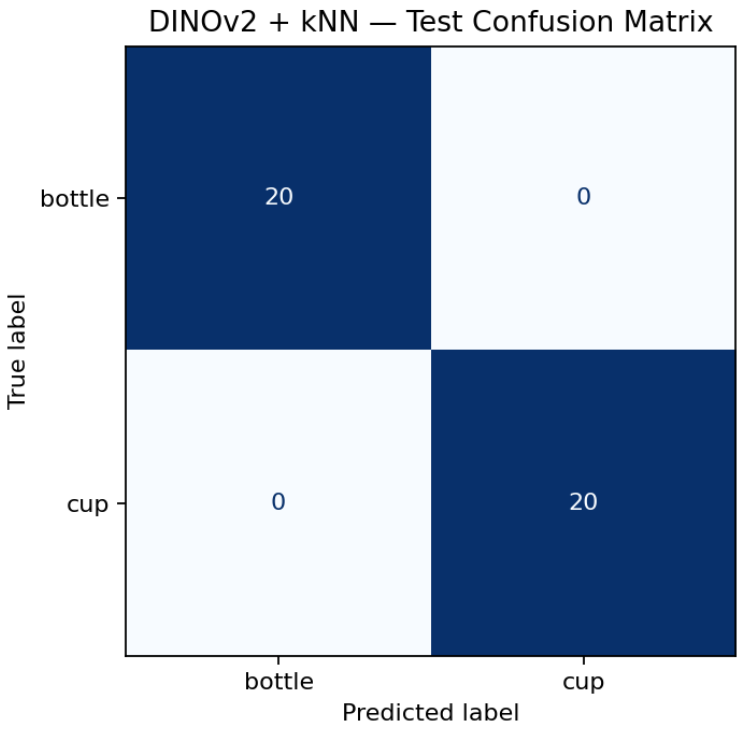


Figure 3. Rows show true categories and columns show predicted categories for the new DINOv2 model.

Evaluation definitions

Accuracy is the fraction of all correct predictions. Precision quantifies how many predictions of a category are correct. Recall measures how many true examples of a category are found. F1 is the harmonic mean of precision and recall. Because the dataset has two balanced classes, we report macro precision, recall and F1 alongside accuracy.

Training curves

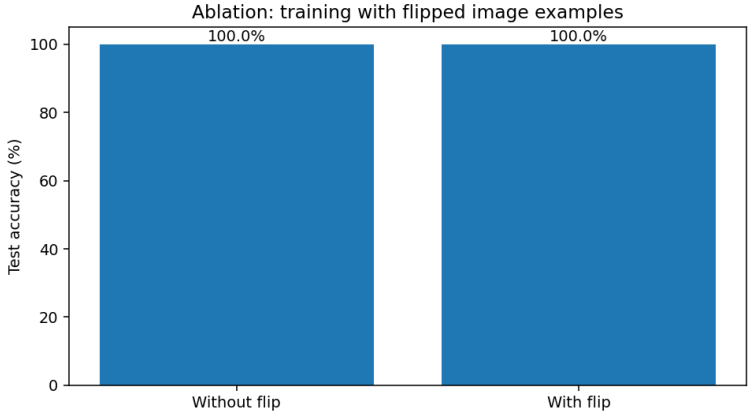
This feature-extraction experiment does not optimize the DINOv2 backbone through training epochs; therefore it does not produce an epoch-wise deep-learning loss curve. The user can upload assignment1_results.zip while executing the notebook to include the real MobileNetV2 training history from Assignment 1.

5. Ablation, robustness and explainability

Ablation design

We fix the DINOv2 backbone, data split, cosine-distance classifier and the validation-selected value of k. The sole changed component is whether every training image has an additional horizontally flipped embedding. This tests a simple augmentation strategy without leaking test images into the training index.

Ablation	Test accuracy	Macro F1
No flip	100.00%	100.00%
Add flipped training views	100.00%	100.00%



Blur robustness experiment

The final augmented k-NN model scored 100.00% on unmodified test images and 97.50% when the test images were blurred using a Gaussian filter with radius 2. This represents a controlled disturbance, not synthetic image generation.

Example-based visual explanation



Figure 5. One test photo and three similar labeled training photographs according to DINOv2 embedding similarity. Similarity is illustrative evidence, not proof of the model's reasoning.

6. Web demo, ethics and reproducibility

Application and measured latency

The Gradio application accepts user-provided photos and webcam snapshots. The program performs the same preprocessing as the experiment, embeds the image with DINOv2, classifies it with cosine k-NN, shows the nearest three training photos, and measures end-to-end prediction latency in milliseconds on the current CPU or GPU. The app is not an object detector and does not output bounding boxes.

Ethics, consent and limitations

The source images are public rather than self-collected; their provenance must be disclosed. No generated or synthetic media are used, so no watermark for synthetic content is applicable. Camera use and any personally identifying images require permission and responsible handling. A user should avoid uploading private photographs to publicly exposed demos. The dataset is small (40 test images), drawn from limited image sources, and may not reflect cluttered real scenes, new backgrounds or different camera devices. Nearest-neighbor explanations can be misleading when two objects share background or appearance.

Reproduction guide

1. Open the provided notebook in Google Colab and select a GPU if possible. 2. Upload dataset.zip, optionally upload assignment1_results.zip. 3. Execute all cells in order. 4. The notebook downloads pretrained DINOv2 weights from the official Meta repository and evaluates features on exactly the preserved original split. 5. Download Assignment2_Gradio_Local_App.zip and execute pip install -r requirements.txt; python app.py. 6. Open the local Gradio URL, upload a new photo and record a 3–5 minute walkthrough. 7. Archive the notebook code and report in a repository for reproducibility.